Transaction Management - Study Notes

1. What a transaction is

A transaction is a group of operations that must succeed or fail as one unit. A bank transfer is the standard example: it debits one account and credits another. If only the debit is applied the money disappears.

Transactions are started with BEGIN, committed with COMMIT and undone with ROLLBACK. A database may also end a transaction implicitly.

2. ACID properties

Atomicity means all operations in the transaction are completed or none are. The database guarantees this with an undo log.

Consistency means the transaction moves the database from one valid state to another valid state, respecting all constraints.

Isolation means the effect of a transaction is not visible to other transactions until it commits. Weak isolation allows anomalies.

Durability means once a transaction commits, the result survives crashes, which is why the database writes to a log and flushes it to disk before it reports success.

3. Schedules and anomalies

A schedule is the order in which operations of concurrent transactions execute. Serial schedules are correct but slow. Interleaved schedules are faster but can produce anomalies.

A dirty read happens when a transaction reads a value that another transaction later rolls back, so the reader acted on data that never officially existed. A non-repeatable read happens when a transaction reads the same row twice and gets different values because another transaction committed in between. A phantom read happens when a transaction repeats a range query and sees new rows that match the condition. A lost update happens when two transactions read the same value and only one of the writes survives.

4. Isolation levels

Read uncommitted allows dirty reads. Read committed prevents dirty reads but allows non-repeatable reads. Repeatable read prevents dirty and non-repeatable reads but can allow phantoms. Serializable prevents all of them and matches serial execution.

Higher isolation reduces concurrency but keeps the database correct, so the practical goal is the weakest level that still prevents the anomalies an application cannot tolerate.

5. Locking, two-phase locking and MVCC

Two-phase locking has two phases. In the growing phase a transaction may take locks but not release them. In the shrinking phase it may release locks but not take new ones. A transaction that follows two-phase locking is conflict serializable, and it cannot deadlock only if it is also wound wait.

Shared locks allow several readers and block writers. Exclusive locks allow only one transaction. Lock upgrade is the move from shared to exclusive, and it can deadlock.

Multi-version concurrency control, MVCC, is an alternative to locking. It keeps several versions of each row and lets each transaction read the version that was current when the transaction started, so readers never block writers and never see uncommitted changes.

6. Deadlocks

A deadlock is a cycle in which every transaction is waiting for a lock held by another transaction in the cycle. Two transactions that each lock two rows in opposite order deadlock reliably.

Deadlocks are detected either by waiting, by timing out, or by building a wait-for graph and looking for a cycle. The standard responses are to abort one victim transaction so it can be retried, or to prevent deadlocks by always acquiring locks in a consistent global order.